

# Guia de Actividad - Completar rutas POST y PUT en el Backend

Task Manager Backend - Node.js + Express + TypeScript

**Objetivo:** completar dos rutas pendientes del backend: **POST /tasks** para crear tareas y **PUT /tasks/:id** para actualizar o marcar una tarea como completada.

Esta actividad continúa desde el servidor Express ya creado, donde ya existen las rutas **GET /** y **GET /tasks**.

## 1. Antes de empezar

Asegurate de que tu backend ya tenga esta base en **backend/src/index.ts**:

```
const express = require("express");

const app = express();
const PORT = 3000;

type Task = {
  id: number;
  text: string;
  completed: boolean;
};

const tasks: Task[] = [
  { id: 1, text: "Estudiar Node.js", completed: false },
  { id: 2, text: "Crear servidor Express", completed: true },
  { id: 3, text: "Probar rutas del backend", completed: false }
];

app.get("/", (req: any, res: any) => {
  res.send("Backend is working!");
});

app.get("/tasks", (req: any, res: any) => {
  res.json(tasks);
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

## 2. Actividad 1 - Implementar POST /tasks

**Que hace esta ruta:** recibe el texto de una nueva tarea desde el cliente, crea un objeto nuevo y lo agrega al arreglo **tasks**.

Primero, agrega esta linea despues de **const PORT = 3000;**:

```
app.use(express.json());
```

Esta linea permite que Express pueda leer datos enviados en formato JSON dentro de **req.body**. Sin esta linea, el backend no puede leer correctamente el texto enviado por el cliente.

Luego, debajo de la ruta **GET /tasks**, agrega esta ruta:

```
app.post("/tasks", (req: any, res: any) => {
  const { text } = req.body;

  if (!text || text.trim() === "") {
```

```

        return res.status(400).json({
            message: "Task text is required"
        });
    }

    const newTask: Task = {
        id: Date.now(),
        text: text.trim(),
        completed: false
    };

    tasks.push(newTask);

    res.status(201).json(newTask);
});

```

## Explicacion breve del codigo

| Codigo                    | Significado                                                |
|---------------------------|------------------------------------------------------------|
| req.body                  | Contiene los datos enviados por el cliente.                |
| const { text } = req.body | Extrae el texto de la nueva tarea.                         |
| status(400)               | Indica que la solicitud esta mal porque falta informacion. |
| Date.now()                | Genera un id temporal para la nueva tarea.                 |
| tasks.push(newTask)       | Agrega la nueva tarea al arreglo en memoria.               |
| status(201)               | Indica que el recurso fue creado correctamente.            |

## Como probar POST /tasks

Esta ruta no se prueba escribiendo la URL en el navegador. Debes usar Postman o Thunder Client.

Method: POST  
URL: <http://localhost:3000/tasks>  
Body: JSON

```

{
  "text": "Aprender POST en Express"
}

```

Respuesta esperada:

```

{
  "id": 1760000000000,
  "text": "Aprender POST en Express",
  "completed": false
}

```

Luego prueba **GET <http://localhost:3000/tasks>** y verifica que la nueva tarea aparezca en la lista.

### 3. Actividad 2 - Implementar PUT /tasks/:id

**Que hace esta ruta:** busca una tarea por su id y permite actualizar su texto o cambiar si esta completada.

Debajo de la ruta **POST /tasks**, agrega:

```
app.put("/tasks/:id", (req: any, res: any) => {
  const id = Number(req.params.id);
  const { text, completed } = req.body;

  const task = tasks.find((task) => task.id === id);

  if (!task) {
    return res.status(404).json({
      message: "Task not found"
    });
  }

  if (text !== undefined) {
    task.text = text;
  }

  if (completed !== undefined) {
    task.completed = completed;
  }

  res.json(task);
});
```

#### Explicacion breve del codigo

| Codigo                  | Significado                                         |
|-------------------------|-----------------------------------------------------|
| req.params.id           | Lee el id que viene en la URL.                      |
| Number(...)             | Convierte el id de texto a numero.                  |
| tasks.find(...)         | Busca la tarea dentro del arreglo.                  |
| status(404)             | Indica que no se encontro la tarea.                 |
| text !== undefined      | Actualiza el texto solo si se envio un nuevo texto. |
| completed !== undefined | Actualiza completed solo si se envio ese valor.     |

#### Como probar PUT /tasks/:id

Tambien se prueba con Postman o Thunder Client.

Method: PUT  
URL: http://localhost:3000/tasks/1  
Body: JSON

```
{
  "completed": true
}
```

Respuesta esperada:

```
{
  "id": 1,
  "text": "Estudiar Node.js",
  "completed": true
}
```

Tambien puedes cambiar el texto:

Method: PUT  
URL: http://localhost:3000/tasks/1  
Body: JSON

```
{
  "text": "Estudiar backend con Express"
}
```

## 4. Codigo final esperado en index.ts

```
const express = require("express");

const app = express();
const PORT = 3000;

app.use(express.json());

type Task = {
  id: number;
  text: string;
  completed: boolean;
};

const tasks: Task[] = [
  { id: 1, text: "Estudiar Node.js", completed: false },
  { id: 2, text: "Crear servidor Express", completed: true },
  { id: 3, text: "Probar rutas del backend", completed: false }
];

app.get("/", (req: any, res: any) => {
  res.send("Backend is working!");
});

app.get("/tasks", (req: any, res: any) => {
  res.json(tasks);
});

app.post("/tasks", (req: any, res: any) => {
  const { text } = req.body;

  if (!text || text.trim() === "") {
    return res.status(400).json({
      message: "Task text is required"
    });
  }

  const newTask: Task = {
    id: Date.now(),
    text: text.trim(),
    completed: false
  };

  tasks.push(newTask);

  res.status(201).json(newTask);
});

app.put("/tasks/:id", (req: any, res: any) => {
  const id = Number(req.params.id);
  const { text, completed } = req.body;

  const task = tasks.find((task) => task.id === id);

  if (!task) {
```

```

        return res.status(404).json({
            message: "Task not found"
        });
    }

    if (text !== undefined) {
        task.text = text;
    }

    if (completed !== undefined) {
        task.completed = completed;
    }

    res.json(task);
});

app.get("/tasks/delete/:id", (req: any, res: any) => {
    const id = Number(req.params.id);
    const taskExists = tasks.some((task) => task.id === id);

    if (!taskExists) {
        return res.status(404).json({
            message: "Task not found"
        });
    }

    const updatedTasks = tasks.filter((task) => task.id !== id);
    tasks.length = 0;
    tasks.push(...updatedTasks);

    res.json({
        message: "Task deleted successfully",
        tasks: tasks
    });
});

app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});

```

## 5. Entrega para la proxima clase

Para la proxima clase, cada estudiante debe mostrar:

- Servidor ejecutando con npm run dev.
- GET http://localhost:3000/tasks funcionando.
- POST http://localhost:3000/tasks creando una nueva tarea.
- PUT http://localhost:3000/tasks/:id actualizando texto o completed.
- Ruta de eliminar funcionando segun lo trabajado en clase.
- Codigo subido a GitHub.

## Nota importante

Los datos todavia estan en memoria. Eso significa que si se reinicia el servidor, las tareas vuelven al arreglo inicial. La persistencia real se trabajara cuando se conecte una base de datos.